

CS504 SHORT NOTES FINAL TERM

LEC 23 TO 45

BY VUONLINEHELP.BLOGSPOT.COM

YOUTUBE LINK:

WEBSITE LINK:

[HTTPS://VUONLINEHELP.BLOGSPOT.COM/](https://vuonlinehelp.blogspot.com/)

IMPORTANT NOTE

You should never copy paste the same answer which we are providing. Please do make some changes in the solution file. If you appreciate our work, please share our website vuonlinehelp.blogspot.com with your friends.

آپ کو کبھی بھی وہی جواب کاپی پیسٹ نہیں کرنا چاہیے جو ہم فراہم کر رہے ہیں۔ براہ کرم حل فائل میں کچھ تبدیلیاں کریں۔ اگر آپ ہمارے کام کی تعریف کرتے ہیں تو براہ کرم ہماری ویب سائٹ vuonlinehelp.blogspot.com کو اپنے دوستوں کے ساتھ شیئر کریں۔

Lecture No. 23

Architectural Views

Krutchen proposed that the software architecture model should be composed of multiple views as a software architecture deals with abstraction, with decomposition and composition, with style and esthetics as well as requirements from different stake holders. His architectural model is known as **Krutchen's 4+1 architectural view model**. As evident, this model proposes the development of 5 main views namely.

Logical view: The logical view is the object model of the design.

Process view: The process view captures the concurrency and synchronization aspects of the design.

Physical view: The physical view documents the mapping(s) of the software onto the hardware and reflects its distributed aspect.

Development view: The development view describes the static organization of the software in its development environment.

Use case view: The use case view uses selected use cases or scenarios to validate that the architecture supports the required functionality.

Views

Views are an engineering tool to help achieve desired system qualities. Each view provides an engineering handle on certain quality attributes. In some systems, distinct views collapse into one. Views are also used as documentation vehicle for current development and future development. Users of views include both managers and customers. For these reasons views must be annotated to support analysis. This annotation can be achieved with the help of scenarios and design rationale.

They are also used to understand the dynamic behavior of the system and understand the limits of design.

View	Components	Users	Rationale
Functional View	functions, key system abstractions, domain elements	domain engineers, product-line designers, end users	functionality, modifiability, product lines/reusability, tool support, work allocation
Code View	classes, objects, procedures, functions, subsystems, layers, modules	programmers, designers, reusers	modifiability/maintainability, portability, subsetability
Development View	files, directories	managers, programmers, configuration managers	managers, programmers, configuration managers
Physical View	CPUs, sensors, storage	hardware engineers, system engineers	system delivery and installation, performance, availability, scalability, security
Concurrency View	processes, threads	performance engineers, integrators, testers	performance, availability

Hierarchical Views

Every view is potentially hierarchical, e.g. functional view could contain sub-functions. Similarly development view contains directories which contain files. Code view would have modules and systems that contain sub-modules and sub-systems respectively. Concurrency view contains processes that are further subdivided into threads. Finally, physical view clusters contain computers which contain processors.

Architectural styles

The architectural model of a system may conform to a generic architectural model or style. An awareness of these styles can simplify the problem of defining system architectures. However, most large systems are heterogeneous and do not follow a single architectural style.

Each style describes a system category that encompasses:

- (1) A set of components (e.g., a database, computational modules) that perform a function required by a system.
- (2) A set of connectors that enable “communication, coordination and cooperation” among components.
- (3) Constraints that define how components can be integrated to form the system and semantic models that enable a designer to understand the overall properties of a system by analyzing the known properties of its constituent parts.

Architectural models

Static structural model shows the major system components while a dynamic process model shows the process structure of the system. Interface models are developed to define sub-system interfaces.

Architectural Styles

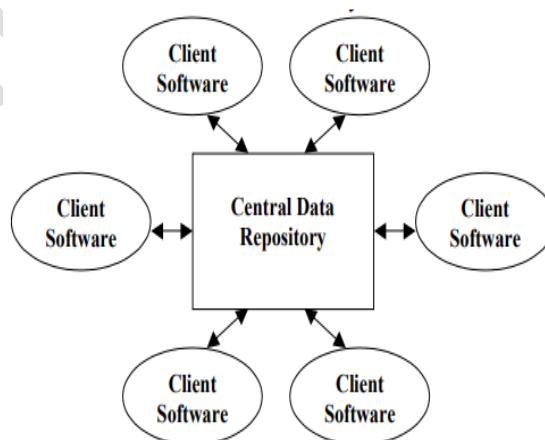
Architectural Styles Architectural design may be based upon a certain pattern of model. These different patterns are also called architectural styles. These styles have different characteristics and attributes and can be useful to solve problems related to a particular situation of requirement. Among the many styles, the most commonly practiced are the following:

- Data-centered architectures
- Client Server Architecture and its variations
- Layered architectures
- Reference Architecture

Data-Centered or the repository model

In any system, sub-systems need to exchange information and data. This may be done in two ways:

1. Shared data is held in a central database or repository and may be accessed by all subsystems
2. Each sub-system maintains its own database and passes data explicitly to other subsystems.



Repository model characteristics

➤ **Advantages**

Repository model is an efficient way to share large amounts of data. In this case subsystems need not be concerned with how data is produced Centralised management e.g. backup, security, etc. This model also provides a global view of the system and the sharing model is published as the repository schema.

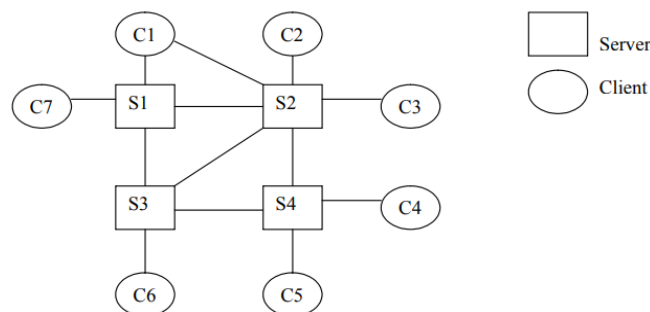
➤ **Disadvantages**

subsystems must agree on a repository data model. This inevitably leads to a compromise. The major problem however is that data evolution is difficult and expensive. There is also little scope for implementing specific management policies. It is also difficult to distribute efficiently.

Client-server model

Client server model tries to distribute data and processing. This is a shift from mainframe based applications where both the data management and the processing of data used to be typically carried out by the same main-frame computer. The client-server model is a distributed system model which shows how data and processing is distributed across a range of components. In this model, the application is modeled as a set of services that are provided by servers and a set of clients that use these services. These clients and servers are connected through a network which allows clients to access servers. Clients and servers are logical processes (not always physical machines).

The following diagram depicts a general client-server organization.



Client/Server Software Components

A typical client-server architecture based system is composed of a number of different components. These include user interaction/presentation subsystem, application subsystem, database management subsystem, and middleware. In this case the application components may reside on either client or server side. Middleware provides the mechanism and protocols to connect clients with the servers.

Representative Client/Server Systems

Following are some of the representative server types in a client-server system.

- **File servers:** In this case, client requests selected records from a file and the server transmits records to client over the network.
- **Database servers:** In this case, client sends requests, such as SQL queries, to the database server; the server processes the request and returns the results to the client over the network.
- **Transaction servers:** In this configuration, client sends requests that invoke remote procedures on the server side; server executes procedures invoked and returns the results to the client.
- **Groupware servers:** Groupware servers provide set of applications that enable communication among clients using text, images, bulletin boards, video, etc.

Client-server characteristics

Advantages:

The main advantage of the client-server architecture is that it makes effective use of networked systems. Instead of building the system with very expensive large computers and hardware devices, cheaper hardware may be used to gain performance and scalability. In addition, adding new servers or upgrading existing servers becomes easier. Distribution of data is straightforward in this case.

Disadvantages:

The main disadvantage of this model is that there is no standard way of sharing data, so sub-systems may use different data organisation. Hence data interchange may be inefficient. From a management point of view, each server needs attention and hence there is redundant management in each server. Finally there is no central register of names and services - it may be hard to find out what servers and services are available.

Representative Client/Server Configurations

The client-server model can have many different configurations. In the following sections, we look at some of these configurations.

Thin Client Model:

In computer networking, a thin client is a simple (low-performance) computer that has been optimized for establishing a remote connection with a server-based computing environment. They are sometimes known as network computers or in their simplest form as zero clients.

Fat Client Model:

A fat client is a networked computer with many locally-stored programs or resources and little dependence on network resources, such as auxiliary drives, CD-RW/DVD players or software applications. Typically, users prefer fat client computers over thin clients because fat clients allow easy customization and greater control over installed programs and system configuration.

Lecture No. 25

Zero Install

Zero Install is a decentralised cross-distribution software installation system. Other features include full support for shared libraries (with a SAT solver for dependency resolution), sharing between users, and integration with native platform package managers.

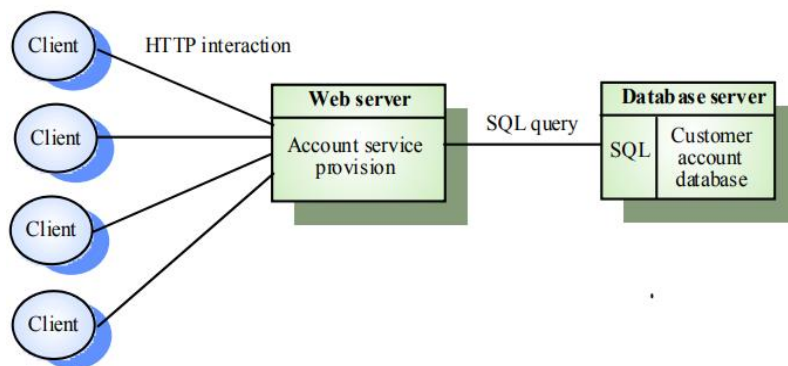
N-Tier architecture

N-tier architecture stems from the struggle to find a middle ground between the fat client architecture and the thin-client architecture. In this case the idea is to enhance scalability and performance by distributing both the data and the application using multiple server machines.

Three-tier Architecture

In this architecture, each application architecture layers (presentation, application, and database) may run on separate processors. It therefore allows for better performance than a thin-client approach. It is simpler to manage than fat client approach and highly scalable (as demands increase add more servers).

Typical 3-tier architecture is depicted in the following diagram.

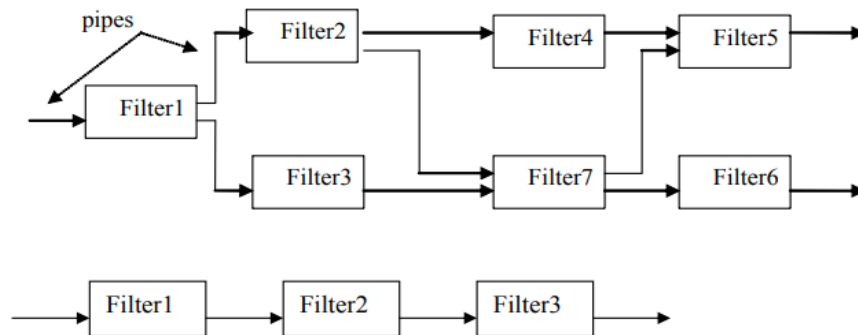


Data Flow or Pipes and Filters Architecture

This architecture is very similar to data flow diagrams. This is used when the input data is processed through a series of transformations to yield the desired output. It is also known as pipes and filters architecture where each processing step is called a filter and the connecting link

from one process to the other is called the pipe through which the information flows from one process to the other.

If the dataflow has only a single sequence of processes with no alternative or parallel paths, then it is called **batch sequential**. These models are depicted in the following diagrams.

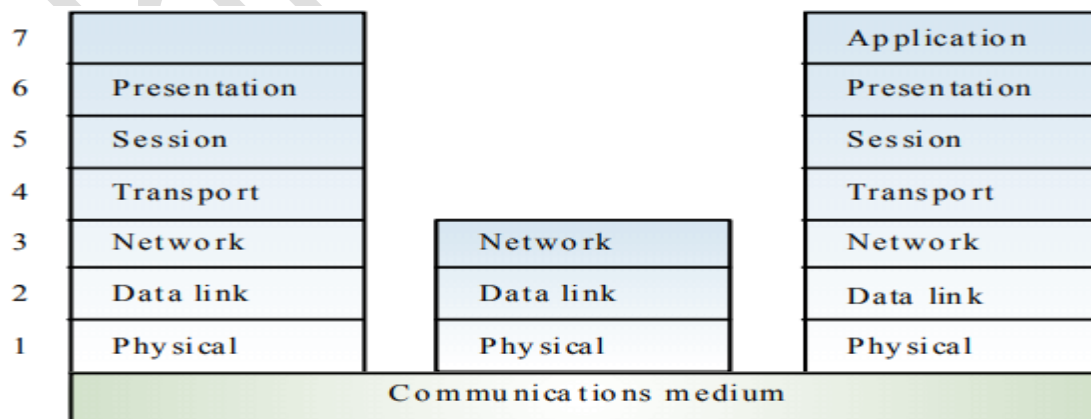


Layered Architecture

Layered architecture patterns are n-tiered patterns where the components are organized in horizontal layers. This is the traditional method for designing most software and is meant to be self-independent. This means that all the components are interconnected but do not depend on each other.

Reference architectures

Reference architecture is a document or set of documents that provides recommended structures and integrations of IT products and services to form a solution. The reference architecture embodies accepted industry best practices, typically suggesting the optimal delivery method for specific technologies.



Partitioning the Architecture

Partitioning of architecture is an important concept. What we basically want to do is distribute the responsibilities to different subsystems so that we get a software system which is easy to maintain. Partitioning of architecture may be “horizontal” and/or “vertical”. In the horizontal partitioning we define separate branches of the module hierarchy for each major function and control modules are used to coordinate communication between functions. Vertical partitioning divides the application from a decision making perspective. The architecture is partitioned in horizontal layers so that decision making and work are stratified with the decision making modules residing at the top of the hierarchy and worker coming at the bottom.

Analyzing Architecture design

Trade-offs seeks optimal combinations of properties based on cost/benefit analysis. So the analysis requires an understanding of what is required and the relative priority of the attributes has to be established. The following sequence of steps provides a guideline for performing architectural analysis.

1. Collect scenarios.
2. Elicit requirements, constraints, and environment description.
3. Describe the architectural styles/patterns that have been chosen to address the scenarios and requirements. These views include module view, process view, and data flow view.
4. Evaluate quality attributes by considered each attribute in isolation.
5. Identify the sensitivity of quality attributes to various architectural attributes for a specific architectural style.

Lecture No. 26

Design Patterns

Christopher Alexander says, “Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice.” A Pattern Language: Towns/Buildings/Construction, 1977.

Design Patterns defined

“Description of communicating objects and classes that are customized to solve a general design in a particular context.”

Historical perspective of design patterns

The origin of design patterns lies in work done by an architect named Christopher Alexander during the late 1970s. He began by writing two books, A Pattern Language[Alex77] and A Timeless Way of Building [Alex79] which, in addition to giving examples, described his rationale for documenting patterns.

GOF (gang of four) Design pattern format includes :

Name, problem, context, forces, solution , result, rationality , related patterns and known uses of the format being applied .

Patterns can be classified as under:

According to their creation here, we look into how the patterns are created. According to structure here we deal with the object's nature. According to behavior here, we use inheritance to distribute behavior between classes.

Framework:

A framework simply dictates the architecture of the software. In other words, framework consists of re-useable patterns.

Lecture No. 27

Skip

Lecture No. 28

important

Maintainable Code

Maintainability of a code means that the code should be clear, simple and flexible.

Self-Documenting Code

Self-documenting the code means that the code should explain itself without the need of comments and extraneous documentation. These include, the size of each function, choice of variable and other identifier names, style of writing

expressions, structure of programming statements, comments, modularity, and issues relating to performance and portability.

Function Size

The size of individual functions plays a significant role in making the program easy or difficult to understand. In general, as the function becomes longer in size, it becomes more difficult to understand. Ideally speaking, a function should not be larger than 20 lines of code and in any case should not exceed one page in length.

Identifier Names

Identifier names also play a significant role in enhancing the readability of a program. In order to understand the concept, let us look at the following statement.

```
if (x==0) // this is the case when we are allocating a new number
```

```
if (AllocFlag == 0)
```

```
If (AllocFlag == NEW_NUMBER)
```

We have improved the quality of the code by replacing the number 0 with a named constant NEW_NUMBER. hence this piece of code is self-documenting.

Coding Style Guide

Consistency plays a very important role in making it self-documenting. A consistently written code is easier to understand and follow. A coding style guide is aimed at improving the coding process and to implement the concept of standardized and relatively uniform code throughout the application or project. As a number of programmers participate in developing a large piece of code, it is important that a consistent style is adopted and used by all.

Naming Conventions

Hungarian Notation was first discussed by Charles Simonyi of Microsoft. It is a variable naming convention that includes information about the variable in its name (such as data type, whether it is a reference variable or a constant variable, etc).

Lecture No. 29

File handling tips for Java and C++

1. C++ header files should have the extension .h. Source files can have the extension .c++ (recommended), .C, .cc or .cpp.
MyClass.c++, MyClass.h
2. Classes should be declared in individual header files with the file name matching the class name. Secondary private classes can be declared as inner classes and reside in the file of the class they belong to. All definitions should reside in source files.
3. Special characters like TAB and page break must be avoided.
4. The incompleteness of split lines must be made obvious.

Include Files and Include Statements for Java and C++

1. Header files must include a construction that prevents multiple inclusions.

Classes and Interfaces

Class and Interface declarations should be organized in the following manner:

1. Class/Interface documentation.
2. class or interface statement.
3. Class (static) variables in the order public, protected, package (no access modifier), private.
4. Instance variables in the order public, protected, package (no access modifier),
5. private.
6. Constructors.
7. Methods (no specific order).

Statements in Java and C++

1. Type conversions must always be done explicitly.
`floatValue = (float) intValue; // NOT: floatValue = intValue;`

2. Types that are local to one file only can be declared inside that file.
3. The parts of a class must be sorted public, protected and private. All sections must be identified explicitly.

Variables

Variables should be initialized where they are declared and they should be declared in the smallest scope possible. Variables must never have dual meaning. This enhances readability by ensuring all concepts are represented uniquely. Reduce chance of error by side effects.

Loop structures

Only loop control statements must be included in the for() construction.

```
sum = 0; // NOT: for (i=0, sum=0; i<100; i++)  
for (i=0; i<100; i++) // sum += value[i];  
sum += value[i];
```

Loop variables should be initialized immediately before the loop.

```
boolean done = false; // NOT: boolean done = false;  
while (!done) { // :  
: // while (!done) {  
} // :  
}
```

Conditionals

Complex conditional expressions must be avoided. Introduce temporary boolean variables instead.

```
if ((elementNo < 0) || (elementNo > maxElement) ||  
elementNo == lastElement) {  
:  
}
```

Should be replaced by:

```
boolean isFinished = (elementNo < 0) || (elementNo > maxElement);  
boolean isRepeatedEntry = elementNo == lastElement;
```

```
if (isFinished || isRepeatedEntry) {  
:  
}
```

The nominal case should be put in the if-part and the exception in the else-part of an

```
if statement.  
boolean isError = readFile (fileName);  
if (!isError) {  
:  
}  
else {  
:  
}
```

Lecture No. 30

important

Comments

Comments should not be used as deodorants. Tricky code should not be commented but rewritten. In general, the use of comments should be minimized by making the code self-documenting by appropriate name choices and an explicit logical structure. There is a need to write comments for whatever reason, the following guidelines should be observed.

1. All comments should be written in English. In an international environment English is the preferred language.
2. Use // for all comments, including multi-line comments. // Comment spanning // more than one line

Since multilevel commenting is not supported in C++ and Java, using // comments ensure that it is always possible to comment out entire sections of a file using /* */ for debugging purposes etc.

Basic indentation should be 2.

```
for (i = 0; i < nElements; i++)  
a[i] = 0;  
Lec 11 :
```

Expression should be written as if they are written as comments or spoken out aloud.
Parenthesize to remove ambiguity

Parentheses should always be used as they reduce complexity and clarify things by specifying grouping.

Breakup complex expressions

Complex expressions should be broken down into multiple statements. An expression is considered to be complex if it uses many operators in a single statement.

```
*x += (*xp=(2*k < (n-m) ? c[k+1] : d[k--]));
```

Shortcuts and cryptic code

Sometimes the programmers, in their creative excitement, try to write very concise code by using shortcuts and playing certain kinds of tricks. This results in a code which is cryptic in nature and hence is difficult to follow.

Lecture No. 31

important

Switch Statement

In the switch statement, cases should always end with a break. A tricky sequence of fallthrough code like the one below causes more trouble than being helpful.

```
switch(c) {  
  case '-': sign = -1;  
  case '+': c = getchar();  
  case '.': break;  
  default : if (! isdigit(c))  
    return 0;  
}
```

This code is cryptic and difficult to read. It is much better to explicitly write what is happening, even at the cost of duplication.

```
switch(c) {  
  case '-': sign = -1;  
    c = getchar();  
    break;  
  case '+': c = getchar();  
    break;
```

```
case '.': break;
default: if (!isdigit(c))
return 0;
break;
}
```

Identified explicitly

There are many more tips like that in the handouts so I suggest you should have a look at the handouts as well.

What do you ment by application server?

An application server is a server that provides software applications with services such as security, data services, transaction support, load balancing, and management of large distributed systems.

Magic Numbers

These are constant that mean something but they do not give any indication of their importance or derivation, making the program hard to understand and modify. To a reader they work like magic and hence are called magic numbers. It is much better to use symbols to explicitly indicate the intent of the statement.¹⁶

Code example of High Coupling

Consider a simple shopping cart application that uses a Cart Contents class to keep track of the items in the shopping cart and an Order class for processing a purchase. The Order needs to determine the total value of the contents in the cart, it might do that like so:

```
public class CartEntry
L{ ec 12 :
public float Price;
public int Quantity;
}
public class CartContents
{
```

```

public CartEntry[] items;
}
public class Order
{
private CartContents cart;
private float salesTax;
public Order(CartContents cart, float salesTax)
{
this.cart = cart;
this.salesTax = salesTax;
}
public float OrderTotal()
{
float cartTotal = 0;
for (int i = 0; i < cart.items.Length; i++)
{
cartTotal += cart.items[i].Price * cart.items[i].Quantity;
}
cartTotal += cartTotal*salesTax;
return cartTotal;
}
}

```

Use (or abuse) of Zero

The number 0 is the most abused symbol in programs written in C or C++. One can easily find code segment that 0 in a fashion similar to the examples below in almost every C/C++ program.

```

flag = 0; // flag is boolean
str = 0; // str is string
name[i] = 0; // name is char array
x = 0; // x is floating pt
i = 0; // i is integer

```

Lecture No. 32

Clarity through modularity

Mentioned earlier, abstraction and encapsulation are two important tools that can help in managing and mastering the complexity of a program. Modularity is a tool

that can help us in reducing the size of individual functions, making them more readable. As an example, consider the following selection sort function.

```
void selectionSort(int a[], int size)
{
    int i, j;
    int temp;
    int min;
    for (i = 0; i < size-1; i++){
        min = i;
        for (j = i+1; j < size; j++){
            if (a[j] < a[min])
                min = j;
        }
        temp = a[i];
        a[i] = a[min];
        a[min] = temp;
    }
}
```

Reusability

Reusability is one of the prime reasons to make functions but is not the only reason. Modularity is of equal concern (if not more) and a function should be broken into smaller pieces, even if those pieces are not reused.

Short circuiting || and &&

The logical and operator, &&, and logical or operators, ||, are special due to the C/C++ short circuiting rule, i.e. $a \parallel b$ and $a \&\& b$ are short circuit evaluated. That is, logical expressions are evaluated left to right and evaluation stops as soon as the final truth value can be determined.

```
struct Node {
    int data;
    Node * next;
};
Node *ptr;
...
```

```
while (ptr->data < myData && ptr != NULL){  
    // do something here  
}
```

Operand Evaluation Order and Side Effects

A side effect of a function occurs when the function, besides returning a value, changes either one of its parameters or a variable declared outside the function but is accessible to it. consider the following statement:

```
c = f1(a) + f2(b);
```

The question is, which function (f1 or f2) will be evaluated first as the C/C++ language does not specify the evaluation order and the implementer (compiler writer) is free to choose one order or the other.

To understand this, let's look at the definition of f1 and f2.

```
int f1(int &x)  
{  
    x = x * 2;  
    return x + 1;  
}  
int f2(int &y)  
{  
    y = y / 2;  
    return x - 1;  
}
```

In this case both f1 and f2 have side effects as they both are doing two things – changing the value of the parameter and changing the value at the caller side. Now if we have the following code segment,

```
a = 3;  
b = 4;  
c = f1(a) + f2(b);
```

then the value of a, b, and c would be as follows:

```
a = 6  
b = 2  
c = 8
```

So far there seem to be any problem. But let us now consider the following statement:

```
c = f1(a) + f2(a);
```

What will be the value of a and c after this statement?

If f1 is evaluated before f2 then we have the following values:

```
a = 3
```

```
b = 9 // 7 + 2
```

On the other hand, if f2 is evaluated before f2 then, we get totally different results.

```
a = 2
```

```
b = 3 // 3 + 0
```

Lecture No. 33

Common Coding mistakes

Following is short list of common mistakes made due to side-effects.

1. `array[i++] = i`; If i is initially 3, `array[3]` might be set to 3 or 4.
2. `array[i++] = array[i++] = x`;
3. 3.“,” is very dangerous as it causes side effects. Let's look at the following statement:
`int i, j = 0`;

Performance:

In many cases, performance and maintainability are at odds with one another. When planning for performance, one should always remember the 80/20 rule - you spend 80 percent of your time in 20 percent of the code. That is, we should not try to optimize everything.

When a small set (a couple of functions) of functions which use each other is so overwhelmingly the bottleneck, there are two alternatives:

1. use a better algorithm
2. rewrite the whole set

```
for (j = i; j < MAX_FIELD; j++)  
clear(j);
```

This loop clears field before each new input is read. It was observed that it was taking almost 50% of the total time.

code was subsequently modified as shown below:

```
for (j = i; j < maxField; j++)  
clear(j);
```

This reduced the overall execution time by half

Lecture No. 34

LONG important

Portability

If we are careful during writing code, we can make it portable. On the other hand if we write code without portability in mind, we may end-up with a code that is extremely hard to port to other environment.

Stick to the standard

1. Use ANSI/ISO standard C++
2. Instead of using vendor specific language extensions, use STL as much as possible

Program in the mainstream

Although C++ standard does not require function prototypes, one should always write them.

```
double sqrt(); // old style acceptable by ANSI C  
double sqrt(double); // ANSI – the right approach
```

Size of data types

Sizes of data types cause major portability issues as they vary from one machine to the other so one should be careful with them.

```
int i, j, k;  
...  
j = 20000;  
k = 30000;  
i = j + k;
```

Order of Evaluation

Order of evaluation refers to the operator precedence and associativity rules according to which mathematical expressions are evaluated.

Signedness of char

The language does not specify whether char is signed or unsigned.

```
char c;  
// between 0 and 255 if unsigned  
// -128 to 127 if signed  
c = getchar();  
if (c == EOF) ??  
// will fail if it is unsigned  
It should therefore be written as follows:  
int c;  
c = getchar();  
if (c == EOF)
```

Arithmetic or Logical Shift

The C/C++ language has not specified whether right shift >> is arithmetic or logical. In the arithmetic shift sign bit is copied while the logical shift fills the vacated bits with 0. This obviously reduces portability. Interestingly, Java has introduced a new operator to handle this issue. >> is used for arithmetic shift and >>> for logical shift.

Bit Fields

A bit field is a data structure that consists of one or more adjacent bits which have been allocated for specific purposes, so that any single bit or group of bits within the structure can be set or inspected. A bit field is most commonly used to represent integral types of known, fixed bit-width, such as single-bit Booleans.

The meaning of the individual bits within the field is determined by the programmer; for example, the first bit in a bit field (located at the field's base address) is sometimes used to determine the state of a particular attribute associated with the bit field.

Lecture No. 35

Exception handling

Exception handling is a powerful technique that separates error-handling code from normal code. It also provides a consistent error handling mechanism. The idea is to raise some error flag every time something goes wrong.

Raising or throwing an error/ trying to execute a block

The idea is to raise some error flag every time something goes wrong. There is a system that is always on the lookout for this error flag. The previous system calls the error handling code if the error flag has been spotted. The raising of the imaginary error flag is simply called raising or throwing an error. When an error is thrown the overall system responds by catching the error. Surrounding a block of error-sensitive code with exception handling is called trying to execute a block.

```
try {  
    ___...  
    ___...  
    ___throw Exception()  
    ___...  
    ___...  
} catch( Exception e )  
{  
    ___...  
    ___...  
}
```

Exceptions and code complexity

A number of invisible execution paths can exist in simple code in a language that allows exceptions. The complexity of a program may increase significantly if there are exceptional paths in it.

```
String EvaluateSalaryAndReturnName( Employee e)  
{  
    if (e.Title() == "CEO" || e.Salary() > 10000)  
    {  
        cout << e.First() << " " << e.Last() << " is overpaid" << endl;  
    }  
    return e.First() + " " + e.Last();  
}
```

The non-exceptional paths:

The three non-exceptional paths are enumerated as below:

1. if (e.Title() == "CEO" || e.Salary() > 10000)
 - if e.Title() == "CEO" is true then the second part is not evaluated and e.Salary() will not be called.
 - cout will be performed
2. if e.Title() != "CEO" and e.Salary() > 10000

- both parts of the condition will be evaluated
- cout will be performed.

3. if e.Title() != "CEO" and e.Salary() <= 10000

- both parts of the condition will be evaluated
- cout will not be performed.

Exception-Safety:

A function is exception safe if it might throw but do not have any side effects if it does throw and any objects being used, including temporaries, are exception safe and clean-up there resources when destroyed.

Exception Neutral:

A function is said to be exception neutral if it propagates all exceptions to the caller.

Levels of Exception Safety

- Basic Guarantee:
Ensures that temporaries are destroyed properly and there are no memory leaks.
- Strong Guarantee:
Ensures basic guarantee as well as there is full-commit or rollback.
- No-throw Guarantee:
Ensure that a function will not throw.

Exception Safety and Multiple Side-effects

It is difficult and some-times impossible to provide strong exception safety when there are two or more side-effects in one function and these side-effects are not related with each other.

Software testing

Software testing is the process of evaluating and verifying that a software product or application does what it is supposed to do. The benefits of testing include preventing bugs, reducing development costs and improving performance.

What is verification and validation in software?

In software project management, software testing, and software engineering, verification and validation (V&V) is **the process of checking that a software system meets specifications and requirements so that it fulfills its intended purpose**. It may also be referred to as software quality control.

Defect :

A defect is a variance from a desired product attribute. These attributes may involve system specifications well as user expectation.

Software and Defect:

Software and defects go side by side in the software development life cycle.

Software Testing:

The process of examining the software product against its requirements thus it is a process that involves verification of product with respect to its written requirements and conformance of requirements with user needs.

Software testing objective:

The correct approach to testing a scientific theory is not to try to verify it, but to seek to refute the theory. That is to prove that it has errors.

Limitations of testing

- You cannot test a program completely
- We can only test against system requirements
 - May not detect errors in the requirements.
 - Incomplete or ambiguous requirements may lead to inadequate or incorrect testing.
- Exhaustive (total) testing is impossible in present scenario.
- Time and budget constraints normally require very careful planning of the testing effort.
- Compromise between thoroughness and budget.
- Test results are used to make business decisions for release dates.
- Even if you do find the last bug, you'll never know it

- You will run out of time before you run out of test cases
- You cannot test every path
- You cannot test every valid input
- You cannot test every invalid input

What are the results of software testing?

The expected outcome in software testing is the ideal result that should be achieved once a test case has been executed. Results refer to **displayed textual or graphical data, database changes, state changes, links, data transmission, etc.**

Code of the Function

```
bool isStringsEqual(char a[], char b[])
{
    bool result;
    if (strlen(a) != strlen(b))
        result = false;
    else {
        for (int i = 0; i < strlen(a); i++)
            if (a[i] == b[i])
                result = true;
            else result = false;
    }
    return result;
}
```

Analysis of the code

Code analysis is the analysis of source code that is performed without actually executing programs. It involves the detection of vulnerabilities and functional errors in deployed or soon-to-be deployed software.

Test Cases and Test Data

In order to test a software application, it is necessary to generate test cases and test data which is used in the application. Test cases correspond to application functionality such that the tester writes down steps which should be followed to achieve certain functionality. **Thus a test case involves**

- **Input and output specification plus a statement of the function under test.**
- **Steps to perform the function**

- Expected results that the software application produces

Lecture No. 37

important

Testing vs. development

Functional specification document is the starting point, base document for both testing and the development.

➤ **Software testing:**

Software testing is process used to identify the correctness completeness and quality of the developed computer software. It includes a set of activities conducted with the intent of finding errors in software so that it could be corrected before product released to the end users. In simple words software testing is an activity to check whether actual results match the expected results and to ensure that software system is defect free. Software testing is more than just error detection.

➤ **Software development:**

Software development used to refer to the activity of computer programming, which is the process of writing and maintaining the source code, but in a broader sense of the term it includes all that is involved between the conception of the desired software through to the final manifestation of the software, ideally in a planned and structured process.

The Developer and Tester

Development	Testing
Development is a creative activity.	Testing is a destructive activity
Objective of development is to show that the program works.	Objective of testing is to show that the program does not work.

Usefulness of testing

While testing can be useful as an assessment tool, researchers suggests that the actual process of taking a test can also help us to learn and retain new information over the long term and apply it across different contexts. New research explores the nuanced

interactions between testing, memory, and learning and suggests possible applications for testing in educational settings.

Testing phases:

Unit testing, Module testing, Subsystem testing, System testing, Acceptance test, Alpha testing, Beta testing.

Description of testing phases

- **Unit testing:** testing individual components independent of other components.
- **Module testing:** testing a collection of dependent components. A module encapsulates related components so it can be tested independently.
- **Subsystem testing:** testing of collection of modules to discover interfacing problems among interacting modules.
- **System testing:** after integrating subsystems into a system. Testing this system as a whole.
- **Acceptance test:** validation against user expectations. Usually it is done at the client premises.
- **Alpha testing:** acceptance testing for customized projects, in-house testing for products.
- **Beta testing:** field testing of product with potential customers who agree to use it and report problem before system is released for general use.

Black box testing/ Structural testing (white box):

In the following two types of testing activities are discussed.

Black box testing:

Not concerned with how the inputs are transformed into outputs. If the outputs match with the expected results, system is fine otherwise a defect is found.

Structural testing (white box):

we look inside the system and evaluate what it consists of and how is it implemented.

Effective testing

The objective of testing is to discover the maximum number of defects with a minimum number of resources before the system is delivered to the next stage. If this objective is achieved then it is effective testing. That is, testing is effective enough to ensure the product rightness.

Equivalence Classes

We divide a problem domain in different classes. These are called Equivalence Classes.

Lecture No. 38

Important

Equivalence Classes or Equivalence Partitioning

Two tests are considered to be equivalent if it is believed that:

- if one discovers a defect, the other probably will too, and
- If one does not discover a defect, the other probably won't either.

Equivalence Classes or Equivalence Partitioning

Equivalence Partitioning Method is also known as Equivalence class partitioning (ECP). It is a software testing technique or black-box testing that divides input domain into classes of data, and with the help of these classes of data, test cases can be derived. An ideal test case identifies class of error that might require many arbitrary test cases to be executed before general error is observed. In equivalence partitioning; equivalence classes are evaluated for given input conditions. Whenever any input is given, then type of input condition is checked, then for this input conditions, Equivalence class represents or describes set of valid or invalid states.

Guidelines for Equivalence Partitioning:

- If the range condition is given as an input, then one valid and two invalid equivalence classes are defined.
- If a specific value is given as input, then one valid and two invalid equivalence classes are defined.
- If a member of set is given as an input, then one valid and one invalid equivalence class is defined.
- If Boolean no. is given as an input condition, then one valid and one invalid equivalence class is defined.

String matching

Equivalence partitions, we divide the problem in two obvious categories of equal strings and the other one for unequal strings. The problem of finding occurrence(s)

of a pattern string within another string or body of text. There are many different algorithms for efficient searching. Also known as exact string matching, string searching, and text searching.

Basis Code Structures

For structural testing it is important to know about basic coding structures. There are four basic coding structures sequence, if statement, case statement, and while loop. These four basic structures can be used to express any type of code.

➤ **Flow graph notation**

A Control Flow Graph (CFG) is the graphical representation of control flow or computation during the execution of programs or applications. Control flow graphs are mostly used in static analysis as well as compiler applications, as they can accurately represent the flow inside of a program unit.

➤ **Sequence**

a sequence diagram is a Unified Modeling Language (UML) diagram that illustrates the sequence of messages between objects in an interaction. A sequence diagram consists of a group of objects that are represented by lifelines, and the messages that they exchange over time during the interaction.

➤ **If**

Second structural form is the If statement. In the following graph, the first node at the left depicts the if statement and the two nodes next to the first node correspond to the successful case (if condition is true) and unsuccessful case (if condition is false) consecutively.

➤ **Case**

in Case statement, control can take either of several branches (as opposed to only two in If statement.) First node represents the switch statement (C/C++) and nodes in middle correspond to all different cases. Program can take one branch and result into the same instruction.

➤ **While**

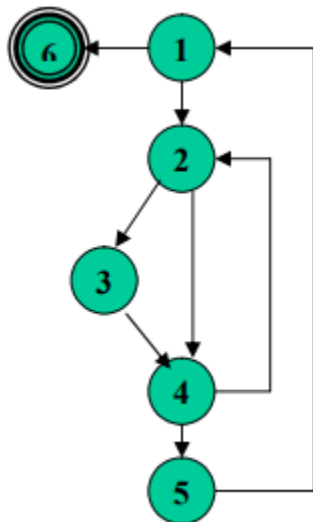
A while loop structure consists of a loop guard instruction through which the iteration in the loop is controlled. The control keeps iterating in the loop as

long as the loop guard condition is true. It branches to the last instruction when it becomes false.

Flow graph for bubble sort

The diagram opposite to the code is the corresponding flow graph model. It consists of six nodes. Node one is the while loop instruction that contains loop guard and node six is the ending instruction of the while loop. Node two corresponds to the for loop instructions. Node three corresponds to the swapping instruction. Nodes five and six correspond to the last instructions of the if statement and the for loop consecutively.

```
sorted = false;  
while (!sorted) { //1  
    sorted = true;  
    for (i=0; i < N-1; i++) { //2  
        if a[i] > a[i+1] {  
            swap(a[i], a[i+1]); //3  
            sorted = false;  
        } //4  
    } //5  
}
```



White box testing

In **white box testing** we test the structure of the program. In this technique the test cases are written in a manner to cover different possibilities in code. Below are described three coverage schemes.

- I. Statement Coverage:
- II. Branch Coverage:
- III. Path Coverage:

Statement Coverage:

In this scheme, statements of the code are tested for a successful test that checks all the statements lying on the path of a successful scenario.

Branch Coverage:

In this scheme, all the possible branches of decision structures are tested. Therefore, sequences of statements following a decision are tested.

Path Coverage:

In path coverage, all possible paths of a program from input instruction to the output instruction are tested. An exhaustive list of test cases is generated and tested against the code.

White Box Testing Example

```
Printme (int a, int b) {  
//          ----- Printme is a function  
    int result = a+ b;  
    If (result> 0)  
        Print ("Positive", result)  
    Else  
        Print ("Negative", result)  
}  
//----- End of the source code
```

Flow graph of a hypothetical program

Following is a flow graph of a hypothetical program whose structure is such that it consists of two loops, one embedded in the other. Left side of the diagram signifies the loops where inner loop seem to be containing two if statements. Second loop has a branch and then program finishes.

Cyclomatic complexity

Cyclomatic Complexity is software metric useful for structured or White Box Testing. It is mainly used to evaluate complexity of a program. If the decision points are more, then complexity of the program is more. If program has high complexity number, then probability of error is high with increased time for maintenance and trouble shoot.

Cyclomatic Complexity, $V(G)$, for a flow graph G is defined as:

$$V(G) = E - N + 2$$

Cyclomatic complexity of selection sort

The following is the code for selection sort. We can draw a control flow graph for this program from the code and each line's corresponding line number.

```
void SelectionSort(int array1[]) {  
    int size = array1.length;  
    for (int i = 0; i < size-1; i++) {  
        int minIndex = i;  
        for (int j = i+1; j < size; j++)  
            if (array1[j] < array1[minIndex])  
                minIndex = j;  
        int temp = array1[minIndex];  
        array1[minIndex] = array1[i];  
        array1[i] = temp;  
    }  
}
```

Infeasible paths

Infeasible path is a path through a program which is never traversed for any input data.

```
if (a == b) //1  
c = c-1; //2  
if (a != b) //3  
c = c+1; //4  
//5
```

A good programming practice is such that minimize infeasible paths to zero.

Lecture No. 40 Unit testing

IMPORTANT

Unit testing

A software program is made up of units that include procedures, functions, classes etc. The unit testing process involves the developer in testing of these units. Unit testing is roughly equivalent to chip-level testing for hardware in which each chip is tested thoroughly after manufacturing.

Software should be tested more like hardware, with

- Built-in self-testing: such that each unit can be tested independently
- Internal diagnostics: diagnostics for program units should be defined.
- Test harness

Unit Testing Principles

In unit testing, developers test their own code units (modules, classes, etc.) during implementation.

- Normal and boundary inputs against expected results are tested.
- Thus unit testing is a great way to test an API.

Quantitative Benefits

- **Repeatable:** Unit test cases can be repeated to verify that no unintended side effects have occurred due to some modification in the code.
- **Bounded:** Narrow focus simplifies finding and fixing defects.
- **Cheaper:** Find and fix defects early

Qualitative Benefits

- **Assessment-oriented:** Writing the unit test forces us to deal with design issues - cohesion, coupling.
- **Confidence-building:** We know what works at an early stage. Also easier to change when it's easy to retest.

Unit Testing Tips

- Start writing tests early
- Write one test per case
- Think more about the intended behavior of the case you're testing
- Think less about arbitrary code coverage targets

- Write your unit tests in a way that makes life easy for the person who will maintain them.
- Maintain your tests like you maintain your production code.
- Automate your testing

Defect origination

In inspections the emphasis is on early detection and fixing of defects from the program. Following are the points in a development life cycle where defects enter into the program.

- Requirements
- Design
- Coding
- User documentation
- Testing itself can cause defects due to bad fixes
- Change requests at the maintenance or initial usage time

It is important to identify defects and fix them as near to their point of origination as possible.

Lecture No. 41

IMPORTANT

Inspection versus Testing

Product testing, often known as lab testing, typically involves testing a product against a specific standard or regulation in a certified laboratory. Whereas product inspection often involves checking a random sample of an order for compliance with a buyer's requirements and specifications.

Inspection pre-conditions

A precise specification must be available before inspections. Team members must be familiar with the organization standards. In addition to it, syntactically correct code must be available to the inspectors. Inspectors should prepare a checklist that can help them during the inspection process.

Inspection checklists

Checklist of common errors in a program should be developed and used to drive the inspection process. These error checklists are programming language dependent such that the inspector has to analyze major constructs of the programming language and develop checklists to verify code that is written using these checklists.

Static analyzers

Static analyzers are software tools for source text processing. They parse the program text and try to discover potentially erroneous conditions and bring these to the attention of the verification and validation team.

Lecture No. 42 Debugging

Debugging

Debugging, in computer programming and engineering, is a multistep process that involves identifying a problem, isolating the source of the problem, and then either correcting the problem or determining a way to work around it. The final step of debugging is to test the correction or workaround and make sure it works.

What is a Bug?

A software bug is an error, flaw, or fault in an application. This error causes the application to produce an unintended or unexpected result, such as crashing or producing invalid results. As software developers, we deal with software bugs all the time.

History of debugging

The use of the word bug as a synonym for error originated in engineering. The term's application to computing and the inspiration for using the word debugging as a synonym for troubleshooting has been attributed to Admiral Grace Hopper, a pioneer in computer programming, who was also known for her dry sense of humor. When an actual bug (a moth) got caught between electrical relays and caused a problem in the U.S. Navy's first computer, Admiral Hopper and her team "debugged" the computer and saved the moth. It now resides in the Smithsonian Museum.

Importance of Debugging:

According to a survey, when a software application is in the maintenance phase, 20% of its lifecycle cost is attributed towards the defects which are found in the software application after installation. Please bear in mind that the maintenance is the phase in which 2/3rd of the overall software cost incurs.

Therefore, 20% of the 2/3rd cost is again a huge cost and we need to understand why this much cost and effort is incurred. In fact, when a software application is installed and being used, any peculiarity in it can cost a lot of direct and indirect damages to the organization.

Problem at AT&T

In the telecommunications industry, loss of service is known as an outage. For most of us, when an outage occurs, we lose our telephone service; we cannot make calls and we cannot receive calls. Outages are accepted and expected hazards in the industry.

The code snippet that caused the outage is illustrated as follows

```
do {  
    ...  
    switch (expression) {  
    case 0: {  
        if (some_condition) {  
            6. ...  
        break;  
        } else {  
    ...  
    }  
    ...  
    break;  
    }  
    ...  
    }  
    ...  
    }  
while (some_other_condition);
```

Art and Science of Debugging

Debugging is taken as an art but in fact it is a scientific process. As people learn about different defect types and come across situations in which they have to debug the code, they develop certain heuristics. Next time they come across a similar situation, they apply those heuristics and solve the problem in lesser time and with a lesser effort.

Program at Bulletin Board Example

Following piece of code was once placed on a bulletin board making an invitation to people to discover the problem in this code.

1. while (i =0; i < 10; i++) {
2. cout << i << endl;
3. }

Lecture No. 43 Bug Classes

Memory and resource leak

A memory leak bug is one in which memory is somehow allocated from either the operating system or an internal memory "pool", but never deallocated when the memory is finished being used.

Symptoms

- System slowdowns
- Crashes that occur "randomly" over a long period of time.

Logical Errors

A logical error occurs when the code is syntactically correct but does not do what you expect it to do.

Symptoms

- The code is misbehaving in a way that isn't easily explained.
- The program doesn't crash, but the flow of the program takes odd branches through the code.
- Results are the opposite of what is expected.
- Output looks strange, but has no obvious symptoms of corruption.

Coding errors

A coding error is a simple problem in writing the code. It can be a failure to check error returns, a failure to check for certain valid conditions, or a failure to take into account other parts of the system. Yet another form of a coding error is incorrect parameter passing and invalid return type coercion.

Symptoms

- Unexpected errors in black box testing.
- The errors that unexpectedly occur are usually caused by coding errors.
- Compiler warnings.
- Coding errors are usually caused by lack of attention to details

Memory over-runs³⁹

A memory overrun occurs when you use memory that does not belong to you. This can be caused by overstepping an array boundary or by copying a string that is too big for the block of memory it is defined to hold. Memory overruns were once extremely common in the programming world because of the inability to tell what the actual size of something really was.

Symptoms

- Program crashes quite regularly after a given routine is called, that routine should be examined for a possible overrun condition.
- If the routine in question does not appear to have any such problem the most likely cause is that another routine, called in the prior sequence, has already trashed variables or memory blocks.
- Checking the trace log of the called routines leading up to one with the problem will often show up the error.

Loop Errors

Loop errors break down into several different subtypes. They occur around a loop construct in a program. Infinite loops, off-by-one loops, and improperly exited loops.

Symptoms

- If your program simply locks up, repeatedly displays the same data over and over, or infinitely displays the same message box, you should immediately suspect an infinite loop error.
- Off-by-one loop errors are quite often seen in processes that perform calculations.
- If a hand calculation shows that the total or final sum is incorrect by the last data point, you can quickly surmise that an off-by-one loop error is to blame.

- Likewise, if you were using graphics software and saw all of the points on the screen, but the last two were unconnected, you would suspect an off-by-one error.
- Watching for a process that terminates unexpectedly when it should have continued.

Pointer errors

A pointer error is any case where something is being used as an indirect pointer to another item. **Uninitialized pointers:** These are pointers that are used to point at something, but we fail to ever assign them something to point at. Deleted pointers, which continue to be used. An Invalid pointer is something that is pointing to a valid block of memory, but that memory does not contain the data you expect it to.

Symptoms

- The program usually crashes or behaves in an unpredictable and baffling way.
- You will generally observe stack corruptions, failure to allocate memory, and odd changing of variable values.
- Changing a single line of code can change where the problem occurs.
- If the problem "goes away" when you place a print statement or new variable into the code that you suspect contains the problem.

Boolean bugs

Boolean bugs occur because the mathematical precision of Boolean algebra has virtually nothing to do with equivalent English words. When we say "and", we really mean the Boolean "or" and vice versa.

Symptoms

- When the program does exactly the opposite of what you expect it to. For example, you might have thought you needed to select only one entry from a list in order to proceed. Instead, the program will not continue until you select more than one. Worse, it keeps telling you to select only one value.

- For true/false problems, you will usually see some sort of debug output indicating an error in a function, only to see the calling function proceed as though the problem had not occurred.

Lecture No. 44 Holistic approach

Holistic approach

Computer software (or just software) is a general term primarily used for digitally stored data such as computer programs and other kinds of information read and written by computers. Today, this includes data that has not traditionally been associated with computers, such as film, tapes and records.

Software is a lot more than just the Microsoft applications which run on a desktop PC or the enterprise applications which run on servers. Software includes such diverse things as video games, web sites, flash applications, mobile phone ringtones & screen savers, smart phone applications, Google Earth, Wikipedia, and Wii games.

Lecture No. 45

Summary Full book

